September 24, 2026: PostgreSQL 19 Beta 4 Released!

11.2. Index Types

11.2.1. B-Tree
11.2.2. Hash
11.2.3. GiST
11.2.4. SP-GiST
11.2.5. GIN
11.2.6. BRIN

PostgreSQL provides several index types: B-tree, Hash, GiST, SP-GiST, GIN, BRIN, and the extension **bloom**. Each index type uses a different algorithm that is best suited to different types of indexable clauses. By default, the `CREATE INDEX` command creates B-tree indexes, which fit the most common situations. The other index types are selected by writing the keyword `USING` followed by the index type name. For example, to create a Hash index:

```
CREATE INDEX name ON table USING HASH (column);
```

11.2.1. B-Tree

B-trees can handle equality and range queries on data that can be sorted into some ordering. In particular, the PostgreSQL query planner will consider using a B-tree index whenever an indexed column is involved in a comparison using one of these operators:

```
<   <=   =   >=   >
```

Constructs equivalent to combinations of these operators, such as `BETWEEN` and `IN`, can also be implemented with a B-tree index search. Also, an `IS NULL` or `IS NOT NULL` condition on an index column can be used with a B-tree index.

The optimizer can also use a B-tree index for queries involving the pattern matching operators `LIKE` and `~` *if* the pattern is a constant and is anchored to the beginning of the string — for example, `col LIKE 'foo%'` or `col ~ '^foo'`, but not `col LIKE '%bar'`. However, if your database does not use the C locale you will need to create the index with a special operator class to support indexing of pattern-matching queries; see Section 11.10 below. It is also possible to use B-tree indexes for `ILIKE` and `~*`, but only if the pattern starts with non-alphabetic characters, i.e., characters that are not affected by upper/lower case conversion.

B-tree indexes can also be used to retrieve data in sorted order. This is not always faster than a simple scan and sort, but it is often helpful.

11.2.2. Hash

Hash indexes store a 32-bit hash code derived from the value of the indexed column. Hence, such indexes can only handle simple equality comparisons. The query planner will consider using a hash index whenever an indexed column is involved in a comparison using the equal operator:

```
=
```

11.2.3. GiST

GiST indexes are not a single kind of index, but rather an infrastructure within which many different indexing strategies can be implemented. Accordingly, the particular operators with which a GiST index can be used vary depending on the indexing strategy (the *operator class*). As an example, the standard distribution of PostgreSQL includes GiST operator classes for several two-dimensional geometric data types, which support indexed queries using these operators:

```
<<   &<   &>   >>   <<|   &<|   |&>   |>>   @>   <@   ~=   &&
```

(See Section 9.11 for the meaning of these operators.) The GiST operator classes included in the standard distribution are documented in Table 68.1. Many other GiST operator classes are available in the `contrib` collection or as separate projects. For more information see Chapter 68.

GiST indexes are also capable of optimizing “nearest-neighbor” searches, such as

```
SELECT * FROM places ORDER BY location <-> point '(101,456)' LIMIT 10;
```

which finds the ten places closest to a given target point. The ability to do this is again dependent on the particular operator class being used. In Table 68.1, operators that can be used in this way are listed in the column “Ordering Operators”.

11.2.4. SP-GiST

SP-GiST indexes, like GiST indexes, offer an infrastructure that supports various kinds of searches. SP-GiST permits implementation of a wide range of different non-balanced disk-based data structures, such as quadtrees, k-d trees, and radix trees (tries). As an example, the standard distribution of PostgreSQL includes SP-GiST operator classes for two-dimensional points, which support indexed queries using these operators:

```
<<   >>   ~=   <@   <<|   |>>
```

(See Section 9.11 for the meaning of these operators.) The SP-GiST operator classes included in the standard distribution are documented in Table 69.1. For more information see Chapter 69.

Like GiST, SP-GiST supports “nearest-neighbor” searches. For SP-GiST operator classes that support distance ordering, the corresponding operator is listed in the “Ordering Operators” column in Table 69.1.

11.2.5. GIN

GIN indexes are “inverted indexes” which are appropriate for data values that contain multiple component values, such as arrays. An inverted index contains a separate entry for each component value, and can efficiently handle queries that test for the presence of specific component values.

Like GiST and SP-GiST, GIN can support many different user-defined indexing strategies, and the particular operators with which a GIN index can be used vary depending on the indexing strategy. As an example, the standard distribution of PostgreSQL includes a GIN operator class for arrays, which supports indexed queries using these operators:

```
<@   @>   =   &&
```

(See Section 9.19 for the meaning of these operators.) The GIN operator classes included in the standard distribution are documented in Table 70.1. Many other GIN operator classes are available in the `contrib` collection or as separate projects. For more information see Chapter 70.

11.2.6. BRIN

BRIN indexes (a shorthand for Block Range INdexes) store summaries about the values stored in consecutive physical block ranges of a table. Thus, they are most effective for columns whose values are well-correlated with the physical order of the table rows. Like GiST, SP-GiST and GIN, BRIN can support many different indexing strategies, and the particular operators with which a BRIN index can be used vary depending on the indexing strategy. For data types that have a linear sort order, the indexed data corresponds to the minimum and maximum values of the values in the column for each block range. This supports indexed queries using these operators:

```
<   <=   =   >=   >
```

The BRIN operator classes included in the standard distribution are documented in Table 71.1. For more information see Chapter 71.

Submit correction

If you see anything in the documentation that is not correct, does not match your experience with the particular feature or requires further clarification, please use this form to report a documentation issue.

Copyright © 1996-2026 The PostgreSQL Global Development Group